

教学过程管理系统

鸿蒙端 API 使用说明

ArkTS 前端队友调用指南

本文面向页面与组件开发人员。前端只通过统一入口 **Api** 调用服务端，不直接处理 URL、Token、HTTP 请求、JSON 解析和 Flask 错误码。

版本：1.0

日期：2026 年 7 月 13 日

目录

1 文档目的	3
2 前端开始调用前的前提	3
2.1 服务端地址已配置	3
2.2 网络权限已声明	3
2.3 应用启动时已初始化 API	4
3 最基本的调用方式	4
3.1 导入统一入口	4
3.2 调用接口	4
4 ArkTS 请求对象的正确写法	5
5 统一错误处理	6
6 登录和会话	6
6.1 登录	6
6.2 获取当前用户	7
6.3 修改密码	7
6.4 退出登录	7
6.5 恢复登录状态	7
7 分页结果和枚举	8
7.1 分页结果	8
7.2 使用枚举，不手写状态字符串	8
8 各业务模块使用示例	9
8.1 个人中心: Api.profile	9
8.2 基础数据和课表: Api.common	9
8.3 课程和选课: Api.course	10
8.4 教室申请: Api.booking	11
8.5 学生请假: Api.leave	11
8.6 教师调课: Api.scheduleChange	12
8.7 通知: Api.notification	13
8.8 人员与教学数据管理: Api.admin	14
9 模块方法速查	15
9.1 Api.auth	15
9.2 Api.profile	15

9.3	Api.common	15
9.4	Api.course	16
9.5	Api.booking	16
9.6	Api.leave	16
9.7	Api.scheduleChange	16
9.8	Api.notification	16
9.9	Api.admin	16
10	前端开发规范	17
10.1	必须遵守	17
10.2	禁止做法	17
11	推荐的页面调用模板	17
11.1	加载数据	17
11.2	提交操作并刷新	18
11.3	防止重复提交	18
12	常见问题排查	19
13	交付与联调检查清单	19
	附录：核心导入速查	20

1 文档目的

本项目已经在 `entry/src/main/ets/api/` 下完成统一 API 封装。前端页面、组件和状态管理代码只需要调用 `Api.auth`、`Api.course`、`Api.booking` 等业务模块,无须直接使用 `http.createHttp()`。统一调用链如下:

页面或组件 → `Api.xxx.xxx()` → 业务 API 模块 → `ApiClient` → `Flask` → `SQLite`

API 封装层已经统一处理:

- Flask 基础地址和接口路径拼接;
- GET 查询参数编码;
- Bearer Token 自动携带;
- JSON 请求体序列化;
- 后端统一响应解析;
- HTTP 错误和业务错误转换;
- Token 失效后的本地会话清理;
- 登录信息的 Preferences 持久化。

2 前端开始调用前的前提

前端队友开始开发页面前, 只需确认以下事项已经由项目集成人员完成。

2.1 服务端地址已配置

`api/core/ApiConfig.ets` 中的地址应指向运行 Flask 的电脑:

```
export class ApiConfig {
  static readonly BASE_URL: string =
    'http://192.168.1.100:8080/api'

  static readonly CONNECT_TIMEOUT: number = 10000
  static readonly READ_TIMEOUT: number = 15000
}
```

模拟器中的 `127.0.0.1` 表示模拟器自身, 通常不能用于访问电脑上的 Flask。需要使用电脑当前局域网 IPv4 地址。

2.2 网络权限已声明

`entry/src/main/module.json5` 中应包含:

```
"requestPermissions": [  
  {  
    "name": "ohos.permission.INTERNET"  
  }  
]
```

2.3 应用启动时已初始化 API

EntryAbility.ets 已在加载首页前执行：

```
await Api.init(this.context)
```

前端页面和组件不要再次调用 `Api.init()`。

前端不得重复初始化

`Api.init()` 只在应用启动时调用一次。页面只负责调用业务方法,不直接操作 `SessionStore`,也不自行创建 `ApiClient`。

3 最基本的调用方式

3.1 导入统一入口

页面位于 `entry/src/main/ets/pages/` 时, 通常这样导入：

```
import { Api } from '../api/Api'
```

组件目录层级不同, 按实际相对路径调整。

3.2 调用接口

所有接口均为异步方法, 需要使用 `await`：

```
const user = await Api.auth.me()  
const home = await Api.profile.getHome()  
const departments = await Api.common.getDepartments()
```

调用者得到的已经是后端响应中的 `data`, 不需要再判断 `code` 或读取 `data`。

后端原始响应：

```
{  
  "code": 0,  
  "message": "success",  
  "data": {  
    "id": 1,  
    "realName": "张三"  
  }  
}
```

前端调用后直接得到：

```
const user = await Api.auth.me()
console.info(user.realName)
```

4 ArkTS 请求对象的正确写法

ArkTS 对对象字面量的静态类型要求比普通 TypeScript 更严格。推荐先明确声明请求或查询类型，再传给 API。

统一写法

每个请求对象、查询对象都使用 model/ 中的具名接口类型。这样既有字段提示，也能避免对象字面量类型报错。

例如查询课程：

```
import { Api } from '../api/Api'
import { CourseQuery } from '../api/model/CourseModel'
import { CourseSelectionStatus } from '../api/core/ApiTypes'

const query: CourseQuery = {
  page: 1,
  pageSize: 20,
  selectionStatus: CourseSelectionStatus.OPEN,
  keyword: '数据库'
}

const result = await Api.course.getCourses(query)
console.info(`total: ${result.total}`)
```

提交教室申请：

```
import { CreateBookingRequest } from '../api/model/BookingModel'

const request: CreateBookingRequest = {
  classroomId: 3,
  bookingDate: '2026-07-20',
  startSection: 3,
  endSection: 4,
  purpose: '课程研讨',
  participantCount: 30
}

const result = await Api.booking.create(request)
console.info(result.bookingNo)
```

5 统一错误处理

所有网络错误、HTTP 错误和 Flask 业务错误都会转换为 `ApiError`。

```
import { ApiError } from '../api/core/ApiError'

try {
  await Api.course.enroll(10)
} catch (error) {
  if (error instanceof ApiError) {
    console.error(`code=${error.code}`)
    console.error(error.message)

    if (error.isUnauthorized()) {
      console.info('登录状态已失效')
    } else if (error.isForbidden()) {
      console.info('当前账号没有权限')
    } else if (error.isConflict()) {
      console.info('发生重复操作或业务冲突')
    } else if (error.isNetworkError()) {
      console.info('无法连接服务器')
    }
  } else if (error instanceof Error) {
    console.error(error.message)
  }
}
```

常用错误判断如下：

方法	含义
<code>isNetworkError()</code>	网络断开、服务器地址错误、连接超时等未获得 HTTP 响应的情况。
<code>isBadRequest()</code>	请求参数缺失、格式错误或取值非法。
<code>isUnauthorized()</code>	未登录、Token 过期、Token 被撤销。
<code>isForbidden()</code>	已登录，但当前角色没有该操作权限。
<code>isNotFound()</code>	用户、课程、申请等目标数据不存在。
<code>isConflict()</code>	重复选课、教室冲突、重复审批、状态不允许等业务冲突。
<code>isServerError()</code>	Flask、数据库或服务端内部错误。

错误提示由页面决定

API 层不会自动显示 Toast，也不会自动操作页面组件。页面根据 `ApiError.message` 决定使用 Toast、对话框、错误占位页或表单字段提示。

6 登录和会话

6.1 登录

```
const result = await Api.auth.login(
  '20240001',
  '123456'
)

console.info(result.realName)
console.info(result.role)
```

登录成功后，Token、用户 ID、账号、姓名、角色和过期时间会自动写入本地 Preferences。前端不得手动保存 Token。

6.2 获取当前用户

```
const user = await Api.auth.me()
console.info(user.account)
console.info(user.realName)
console.info(user.role)
```

6.3 修改密码

```
import { ChangePasswordRequest } from '../api/model/AuthModel'

const request: ChangePasswordRequest = {
  oldPassword: '123456',
  newPassword: 'NewPassword123',
  confirmPassword: 'NewPassword123'
}

await Api.auth.changePassword(request)
```

6.4 退出登录

```
await Api.auth.logout()
```

无论退出接口是否成功返回，本地会话都会被清除。

6.5 恢复登录状态

应用启动后可先检查本地是否存在会话，再调用 `/auth/me` 让服务端验证 Token：

```
const hasSession: boolean = await Api.hasSession()

if (hasSession) {
  try {
    const user = await Api.auth.me()
    console.info(`session valid: ${user.realName}`)
  } catch (error) {
```

```
    console.info('session invalid')
  }
}
```

注意

`Api.hasSession()` 只能说明本地存在 Token，不代表 Token 一定有效。真正的登录恢复必须调用 `Api.auth.me()` 由 Flask 验证。

7 分页结果和枚举

7.1 分页结果

分页接口统一返回：

```
interface PageResult<T> {
  items: Array<T>
  total: number
  page: number
  pageSize: number
}
```

使用示例：

```
const page = await Api.course.getCourses(query)

const courses = page.items
const total = page.total
const currentPage = page.page
```

字段是 `items`，不是 `list`。

7.2 使用枚举，不手写状态字符串

```
import {
  BookingStatus,
  LeaveType,
  ScheduleChangeType,
  NotificationAudienceType
} from '../api/core/ApiTypes'
```

推荐：

```
if (booking.status === BookingStatus.APPROVED) {
  console.info('approved')
}
```

不推荐：

```
if (booking.status === 'APPROVED') {
```

```
// typo  
}
```

8 各业务模块使用示例

8.1 个人中心: Api.profile

```
const profile = await Api.profile.getMyProfile()  
const home = await Api.profile.getHome()
```

修改允许本人编辑的资料:

```
import { UpdateProfileRequest } from '../api/model/ProfileModel'  
  
const request: UpdateProfileRequest = {  
  phone: '13800138000',  
  email: 'student@example.com'  
}  
  
const updated = await Api.profile.updateMyProfile(request)
```

8.2 基础数据和课表: Api.common

健康检查:

```
const health = await Api.common.health()  
console.info(health.status)
```

基础数据:

```
const departments = await Api.common.getDepartments()  
const classes = await Api.common.getClasses(1)  
const semesters = await Api.common.getSemesters()
```

查询教室:

```
import { ClassroomQuery } from '../api/model/CommonModel'  
import { ClassroomStatus, RoomType } from '../api/core/ApiTypes'  
  
const query: ClassroomQuery = {  
  status: ClassroomStatus.AVAILABLE,  
  roomType: RoomType.MULTIMEDIA,  
  minCapacity: 30  
}  
  
const classrooms = await Api.common.getClassrooms(query)
```

查询指定时间的空闲教室:

```
import { AvailableClassroomQuery } from '../api/model/CommonModel'
```

```
const query: AvailableClassroomQuery = {
  date: '2026-07-20',
  startSection: 3,
  endSection: 4,
  participantCount: 30
}

const classrooms =
  await Api.common.getAvailableClassrooms(query)
```

查询某日个人课表:

```
const timetable =
  await Api.common.getMyTimetable('2026-07-20')
```

8.3 课程和选课: **Api.course**

查询课程列表:

```
import { CourseQuery } from '../api/model/CourseModel'
import { CourseSelectionStatus } from '../api/core/ApiTypes'

const query: CourseQuery = {
  page: 1,
  pageSize: 20,
  selectionStatus: CourseSelectionStatus.OPEN
}

const page = await Api.course.getCourses(query)
```

课程详情与排课:

```
const course = await Api.course.getCourse(10)
const schedules = await Api.course.getCourseSchedules(10)
```

学生选课与退课:

```
await Api.course.enroll(10)
await Api.course.drop(10)
```

学生查询自己的选课记录:

```
import { EnrollmentStatus } from '../api/core/ApiTypes'

const records = await Api.course.getMyEnrollments(
  EnrollmentStatus.SELECTED
)
```

教师查询授课信息:

```
const courses = await Api.course.getTeachingCourses()
```

```
const students = await Api.course.getCourseStudents(10)
const schedules = await Api.course.getTeachingSchedules()
```

8.4 教室申请: Api.booking

提交申请:

```
import { CreateBookingRequest } from '../api/model/BookingModel'

const request: CreateBookingRequest = {
  classroomId: 3,
  bookingDate: '2026-07-20',
  startSection: 5,
  endSection: 6,
  purpose: '班会',
  participantCount: 35
}

const result = await Api.booking.create(request)
```

查询本人申请:

```
import { BookingQuery } from '../api/model/BookingModel'
import { BookingStatus } from '../api/core/ApiTypes'

const query: BookingQuery = {
  status: BookingStatus.PENDING
}

const bookings = await Api.booking.getMyBookings(query)
```

详情、取消和日志:

```
const detail = await Api.booking.getDetail(12)
await Api.booking.cancel(12, '活动取消')
const logs = await Api.booking.getLogs(12)
```

教务审批:

```
const pending = await Api.booking.getPending()
await Api.booking.approve(12, '同意使用')
await Api.booking.reject(13, '该时间段已安排教学')
```

8.5 学生请假: Api.leave

提交请假:

```
import { CreateLeaveRequest } from '../api/model/LeaveModel'
import { LeaveType } from '../api/core/ApiTypes'

const request: CreateLeaveRequest = {
```

```
leaveType: LeaveType.SICK,  
startDate: '2026-07-20',  
endDate: '2026-07-20',  
startSection: 1,  
endSection: 4,  
reason: '身体不适',  
courseIds: [2, 5]  
}  
  
const result = await Api.leave.create(request)
```

本人记录与取消:

```
const leaves = await Api.leave.getMyLeaves()  
const detail = await Api.leave.getDetail(8)  
await Api.leave.cancel(8, '无需继续请假')
```

教师查看相关课程请假:

```
const leaves = await Api.leave.getTeachingLeaves({  
  courseId: 2  
})
```

教务审批:

```
const pending = await Api.leave.getPending()  
await Api.leave.approve(8, '同意')  
await Api.leave.reject(9, '证明材料不足')
```

8.6 教师调课: Api.scheduleChange

提交调课:

```
import {  
  CreateScheduleChangeRequest  
} from '../api/model/ScheduleChangeModel'  
import {  
  ScheduleChangeType  
} from '../api/core/ApiTypes'  
  
const request: CreateScheduleChangeRequest = {  
  scheduleId: 20,  
  changeType: ScheduleChangeType.BOTH,  
  reason: '参加教研会议',  
  newClassroomId: 5,  
  newWeek: 10,  
  newWeekDay: 3,  
  newStartSection: 5,  
  newEndSection: 6  
}
```

```
const result = await Api.scheduleChange.create(request)
```

查询和冲突检查:

```
const changes = await Api.scheduleChange.getMyChanges()
const detail = await Api.scheduleChange.getDetail(6)
const conflicts = await Api.scheduleChange.getConflicts(6)

if (conflicts.hasConflict) {
  console.info(`conflicts: ${conflicts.conflicts.length}`)
}
```

取消与审批:

```
await Api.scheduleChange.cancel(6, '计划有变')
const pending = await Api.scheduleChange.getPending()
await Api.scheduleChange.approve(6, '同意调整')
await Api.scheduleChange.reject(7, '新时间存在冲突')
```

8.7 通知: Api.notification

查询收到的通知:

```
import { NotificationQuery } from '../api/model/NotificationModel'

const query: NotificationQuery = {
  page: 1,
  pageSize: 20,
  unread: true
}

const page = await Api.notification.getReceived(query)
const unreadCount = await Api.notification.getUnreadCount()
```

通知详情与已读:

```
const detail = await Api.notification.getDetail(3)
await Api.notification.markRead(3)
```

教师或教务发布通知:

```
import {
  PublishNotificationRequest
} from '../api/model/NotificationModel'
import {
  NotificationAudienceType
} from '../api/core/ApiTypes'

const request: PublishNotificationRequest = {
  title: '课程调整通知',
  content: '本周课程调整至教学楼 A305。',
```

```
audienceType: NotificationAudienceType.CLASS,  
targetClassId: 3,  
expiresAt: '2026-07-25 23:59:59'  
}  
  
const result = await Api.notification.publish(request)
```

本人发布、管理和撤回:

```
const mine = await Api.notification.getPublishedByMe()  
const managed = await Api.notification.getManageList()  
await Api.notification.withdraw(3)
```

8.8 人员与教学数据管理: Api.admin

管理模块主要供教务人员和管理员使用。权限最终由 Flask 判断。

查询人员:

```
import { UserQuery } from '../api/model/AdminModel'  
import { UserRole, EnabledStatus } from '../api/core/ApiTypes'  
  
const query: UserQuery = {  
  page: 1,  
  pageSize: 20,  
  role: UserRole.STUDENT,  
  status: EnabledStatus.ENABLED,  
  keyword: '张'  
}  
  
const page = await Api.admin.getUsers(query)
```

管理员预建学生账号:

```
import { CreateUserRequest } from '../api/model/AdminModel'  
import {  
  AcademicStatus,  
  UserRole  
} from '../api/core/ApiTypes'  
  
const request: CreateUserRequest = {  
  account: '20260001',  
  initialPassword: '123456',  
  realName: '张三',  
  role: UserRole.STUDENT,  
  departmentId: 1,  
  classId: 2,  
  studentNo: '20260001',  
  admissionDate: '2026-09-01',  
  academicStatus: AcademicStatus.ACTIVE  
}
```

```
const user = await Api.admin.createUser(request)
```

启用、停用和重置密码:

```
await Api.admin.updateUserStatus(20, {
  status: EnabledStatus.DISABLED
})

await Api.admin.resetUserPassword(20, {
  newPassword: 'NewPassword123'
})
```

课程、教室、院系、班级、学期和排课的创建与修改, 均使用 `AdminModel.ets` 中对应的请求类型, 再调用 `Api.admin` 中同名方法。

9 模块方法速查

9.1 Api.auth

- `login(account, password)`: 登录并自动保存会话。
- `me()`: 验证 Token 并获取当前用户。
- `logout()`: 服务端退出并清理本地会话。
- `changePassword(request)`: 修改当前用户密码。

9.2 Api.profile

- `getMyProfile()`: 获取完整个人资料。
- `updateMyProfile(request)`: 修改允许本人编辑的字段。
- `getHome()`: 获取按角色聚合的主页数据。

9.3 Api.common

- `health()`: 服务端健康检查, 不需要登录。
- `getDepartments()`: 查询院系。
- `getClasses(departmentId?)`: 查询班级。
- `getSemesters()`: 查询学期。
- `getClassrooms(query?)`: 筛选教室。
- `getClassroom(classroomId)`: 教室详情。
- `getAvailableClassrooms(query)`: 查询空闲教室。
- `getMyTimetable(date)`: 查询指定日期个人课表。

9.4 Api.course

- getCourses(query?)、getCourse(courseId)、getCourseSchedules(courseId)。
- getMyEnrollments(status?)、enroll(courseId)、drop(courseId)。
- getTeachingCourses()、getCourseStudents(courseId)、getTeachingSchedules(semester?)。

9.5 Api.booking

- create(request)、getMyBookings(query?)、getDetail(bookingId)。
- cancel(bookingId, reason?)、getLogs(bookingId)。
- getPending(query?)、approve(bookingId, comment?)、reject(bookingId, comment)。
- finishExpired(): 批量将过期预约标记为完成, 通常不由普通页面直接调用。

9.6 Api.leave

- create(request)、getMyLeaves(query?)、getDetail(leaveId)、cancel(leaveId, reason?)。
- getTeachingLeaves(query?): 教师查看自己课程相关请假。
- getPending()、approve(leaveId, comment?)、reject(leaveId, comment)。

9.7 Api.scheduleChange

- create(request)、getMyChanges(query?)、getDetail(changeId)。
- getConflicts(changeId)、cancel(changeId, reason?)。
- getPending()、approve(changeId, comment?)、reject(changeId, comment)。

9.8 Api.notification

- getReceived(query?)、getUnreadCount()、getDetail(notificationId)、markRead(notificationId)。
- publish(request)、getPublishedByMe()、getManageList()、withdraw(notificationId)。

9.9 Api.admin

- 人员: getUsers、createUser、getUser、updateUser、updateUserStatus、resetUserPassword。
- 院系与班级: createDepartment、updateDepartment、createClass、updateClass。
- 教室与学期: createClassroom、updateClassroom、createSemester、updateSemester。
- 课程与排课: createCourse、updateCourse、updateCourseStatus、createSchedule、updateSchedule、cancelSchedule。
- 审批历史: getApprovals(query?)。

10 前端开发规范

10.1 必须遵守

1. 页面只导入并使用统一的 `Api` 实例。
2. 请求和查询对象使用 `model/` 中明确声明的接口类型。
3. 状态、角色和业务类型使用 `ApiTypes.ets` 中的枚举。
4. 所有 API 调用使用 `try/catch` 处理失败。
5. 列表接口根据返回类型区分 `Array<T>` 和 `PageResult<T>`。
6. 最终权限以 `Flask` 为准；本地角色只用于控制界面展示。
7. 日期统一使用 `YYYY-MM-DD`，日期时间统一使用 `YYYY-MM-DD HH:mm:ss`。

10.2 禁止做法

```
// Do not create another client.
const client = new ApiClient()

// Do not manually send token from page code.
await getCourses(token)

// Do not directly write endpoint URLs in pages.
await client.get('/api/courses')

// Do not parse ApiResponse again.
if (response.code === 0) {
  return response.data
}
```

统一入口

页面和组件不能直接调用 `ApiClient`、`SessionStore` 或业务模块构造函数，也不能重新创建 `TeachingApi`。整个应用只使用 `Api.ets` 导出的单例 `Api`。

11 推荐的页面调用模板

下面的模板不限定具体页面，可用于列表加载、提交表单和审批按钮事件。

11.1 加载数据

```
import { Api } from '../api/Api'
import { ApiError } from '../api/core/ApiError'
import { CourseInfo, CourseQuery } from '../api/model/CourseModel'

private async loadCourses(): Promise<void> {
```

```
const query: CourseQuery = {
  page: 1,
  pageSize: 20
}

try {
  const page = await Api.course.getCourses(query)
  const items: Array<CourseInfo> = page.items
  console.info(`loaded: ${items.length}`)
} catch (error) {
  if (error instanceof ApiError) {
    console.error(error.message)
  }
}
}
```

11.2 提交操作并刷新

```
private async submitEnrollment(courseId: number): Promise<void> {
  try {
    await Api.course.enroll(courseId)
    await this.loadCourses()
  } catch (error) {
    if (error instanceof ApiError) {
      console.error(error.message)
    }
  }
}
}
```

11.3 防止重复提交

```
private submitting: boolean = false

private async submitBooking(
  request: CreateBookingRequest
): Promise<void> {
  if (this.submitting) {
    return
  }

  this.submitting = true

  try {
    await Api.booking.create(request)
  } finally {
    this.submitting = false
  }
}
```

12 常见问题排查

现象	排查方法
提示 SessionStore 尚未初始化	确认 <code>EntryAbility</code> 在加载首页前执行了 <code>await Api.init(this.context)</code> 。页面不得早于初始化调用接口。
无法连接服务器	检查 <code>Flask</code> 是否运行、 <code>ApiConfig.BASE_URL</code> 是否为电脑 IPv4、 <code>Flask</code> 是否监听 <code>0.0.0.0:8080</code> 、Windows 防火墙和 INTERNET 权限。
返回 401	Token 无效、过期或被撤销。API 会清除本地会话，前端应提示重新登录。
返回 403	当前角色没有该接口权限。界面可以隐藏入口，但最终以 <code>Flask</code> 校验结果为准。
对象字面量编译报错	为请求或查询对象声明明确的接口类型，如 <code>const request: CreateBookingRequest = {...}</code> 。
列表字段取不到	确认接口返回的是数组还是 <code>PageResult</code> 。分页内容字段为 <code>items</code> 。
字段总是 undefined	检查前端 <code>Model</code> 字段是否和 <code>Flask JSON</code> 的 <code>camelCase</code> 字段完全一致。
中文查询异常	不需要手工编码，底层 <code>QueryBuilder</code> 会自动执行 URL 编码。
登录成功但后续未带 Token	检查登录是否通过 <code>Api.auth.login()</code> 完成，而不是绕过封装直接请求登录接口。

13 交付与联调检查清单

前端完成页面后，按以下顺序联调：

- 1. 调用 `Api.common.health()`，确认模拟器可以访问 `Flask`。
- 2. 调用 `Api.auth.login()`，确认登录结果和本地会话保存正常。
- 3. 调用 `Api.auth.me()`，确认 `Token` 自动携带。
- 4. 调用一个带查询参数的 `GET`，例如 `Api.course.getCourses(query)`。
- 5. 调用一个带 `JSON` 请求体的 `POST`，例如 `Api.booking.create(request)`。
- 6. 使用错误账号、无权限账号和冲突数据验证 `ApiError`。
- 7. 退出登录后再次调用鉴权接口，确认得到未登录错误。

最终目标

前端队友只需理解业务方法、请求类型和返回类型。服务器地址、HTTP 方法、Token、统一响应、错误码和连接释放等底层细节均由 API 封装层负责。

附录：核心导入速查

```
// Unified API entry
import { Api } from '../api/Api'

// Unified error
import { ApiError } from '../api/core/ApiError'

// Common enums
import {
  UserRole,
  CourseSelectionStatus,
  EnrollmentStatus,
  BookingStatus,
  LeaveType,
  LeaveStatus,
  ScheduleChangeType,
  NotificationAudienceType
} from '../api/core/ApiTypes'

// Request and response models
import { CourseQuery } from '../api/model/CourseModel'
import { CreateBookingRequest } from '../api/model/BookingModel'
import { CreateLeaveRequest } from '../api/model/LeaveModel'
import {
  CreateScheduleChangeRequest
} from '../api/model/ScheduleChangeModel'
import {
  PublishNotificationRequest
} from '../api/model/NotificationModel'
```